Black-Scholes Greeks from First Principles

Exact analytical sensitivities for options pricing, from no-arbitrage derivation through symbolic PDE solution to deployable Greek functions.

Workflow: Ito's lemma → Black-Scholes PDE → closed-form solution → exact Greeks → near-ATM Taylor expansions → code generation → validation

Requires: Symbolic Math Toolbox™, Financial Toolbox™

1. Geometric Brownian Motion and Ito's Lemma

A stock price S follows geometric Brownian motion under the physical measure:

$$dS = \mu S\,dt + \sigma S\,dW$$

For a twice-differentiable function $V(S,t)$ (option value), Ito's lemma gives:

$$dV = \left(\frac{\partial V}{\partial t} + \mu S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2}\right)dt + \sigma S\frac{\partial V}{\partial S}\,dW$$

Construct a delta-hedged portfolio $\Pi = V - \Delta S$ that eliminates the stochastic term. Setting $\Delta = \partial V/\partial S$ and requiring the portfolio to earn the risk-free rate r yields the Black-Scholes PDE.

```
syms S t r sigma tau positive
syms K positive
syms V(S, t)
dVdt = diff(V, t);
dVdS = diff(V, S);
d2VdS2 = diff(V, S, 2);
```

The Black-Scholes PDE:

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2\frac{\partial^2 V}{\partial S^2} - rV = 0$$

```
BS_PDE = dVdt + r*S*dVdS + sigma^2*S^2/2 * d2VdS2 - r*V;
disp('Black-Scholes PDE (= 0):')
```

Black-Scholes PDE (= 0):

```
disp(BS_PDE)
```

$$\frac{\partial}{\partial t} V(S,t) - r\,V(S,t) + \frac{S^2\,\sigma^2\,\frac{\partial^2}{\partial S^2} V(S,t)}{2} + S\,r\,\frac{\partial}{\partial S} V(S,t)$$

2. Solve the Black-Scholes PDE via Substitution

The standard approach transforms the PDE into the heat equation. We work in time-to-expiry $\tau = T - t$ and use the known solution structure for a European call: $C(S,\tau) = S\,N(d_1) - Ke^{-r\tau}N(d_2)$.

Rather than solve the PDE numerically, verify the closed-form solution satisfies the PDE symbolically.

```
syms d1 d2
assume(tau > 0)
d1_expr = (log(S/K) + (r + sigma^2/2)*tau) / (sigma*sqrt(tau));
d2_expr = d1_expr - sigma*sqrt(tau);
```

The European call price in terms of the cumulative normal $N(\cdot)$:

```
C_BS = S * normcdf(d1_expr) - K*exp(-r*tau) * normcdf(d2_expr);
disp('Black-Scholes call price C(S, tau):')
```

Black-Scholes call price C(S, tau):

```
disp(C_BS)
```

$$-S\left(\frac{\operatorname{erfc}\left(\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2\,\sigma\,\sqrt{\tau}}\right)}{2}-1\right)-\frac{K\operatorname{erfc}\left(\frac{\sqrt{2}\left(\sigma\,\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)}{\sigma\,\sqrt{\tau}}\right)}{2}\right)\mathrm{e}^{-r\,\tau}}{2}$$

Verify the Solution Satisfies the PDE

Substitute $C(S,\tau)$ into the PDE (with $\partial/\partial t = -\partial/\partial\tau$). If the formula is correct, the residual simplifies to exactly zero.

```
PDE_residual = -diff(C_BS, tau) + r*S*diff(C_BS, S) ...
    + sigma^2*S^2/2 * diff(C_BS, S, 2) - r*C_BS;
PDE_check = simplify(PDE_residual);
disp('PDE residual (should be 0):')
```

```
PDE residual (should be 0):
```

```
disp(PDE_check)
```

$$0$$

3. Extract All Greeks as Exact Partial Derivatives

Each Greek is a partial derivative of the option price with respect to a market variable.

First-Order Greeks

```
Delta_call = simplify(diff(C_BS, S));
Theta_call = simplify(-diff(C_BS, tau));
Rho_call   = simplify(diff(C_BS, r));
Vega_call  = simplify(diff(C_BS, sigma));
disp('Delta (dC/dS):')
```

```
Delta (dC/dS):
```

```
disp(Delta_call)
```

$$\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}}{2\sigma\sqrt{\tau}\sqrt{\pi}} - \frac{\operatorname{erfc}\left(\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2\sigma\sqrt{\tau}}\right)}{2} - \frac{\sqrt{2}\,K\,\mathrm{e}^{-\frac{\left(\sigma\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)}{\sigma\sqrt{\tau}}\right)^2}{2}}\mathrm{e}^{-r\tau}}{2S\sigma\sqrt{\tau}\sqrt{\pi}} + \frac{1}{2}$$

```
disp('Vega (dC/dsigma):')
```

Vega (dC/dsigma):

```
disp(Vega_call)
```

$$\frac{S\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}\left(\frac{\sqrt{2}\,\sqrt{\tau}}{2}-\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2\sigma^2\sqrt{\tau}}\right)}{\sqrt{\pi}}+\frac{\sqrt{2}\,K\,\mathrm{e}^{-\frac{\left(\sigma\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)}{\sigma\sqrt{\tau}}\right)^2}{2}}\mathrm{e}^{-r}}{2\sigma^2\sqrt{\tau}\,\sqrt{}}$$

```
disp('Theta (-dC/dtau):')
```

Theta (-dC/dtau):

```
disp(Theta_call)
```

$$\frac{K\,r\,\mathrm{e}^{-r\,\tau}\left(\mathrm{erf}\left(\frac{\sqrt{2}\,(\tau\sigma^2+2\,\log(K)-2\,\log(S)-2\,r\,\tau)}{4\,\sigma\,\sqrt{\tau}}\right)-1\right)}{2}-\frac{\sqrt{2}\,S\,\mathrm{e}^{-\frac{(\tau\sigma^2-2\,\log(K)+2\,\log(S)+2\,r\,\tau)^2}{8\sigma^2\tau}}(\tau\sigma^2\cdot}{8\,\sigma\,\tau^{3/2}\,\sqrt{}}$$

```
disp('Rho (dC/dr):')
```

Rho (dC/dr):

```
disp(Rho_call)
```

$$\frac{K\,\tau\,\mathrm{erfc}\left(\frac{\sqrt{2}\left(\sigma\,\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)}{\sigma\,\sqrt{\tau}}\right)}{2}\right)\mathrm{e}^{-r\,\tau}}{2}+\frac{\sqrt{2}\,S\,\sqrt{\tau}\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}}{2\,\sigma\,\sqrt{\pi}}-\frac{\sqrt{2}\,K\,\sqrt{\tau}\,\mathrm{e}^{-\left(\sigma\cdot\right.}}{}$$

Second-Order Greeks

Gamma and Vanna require second partial derivatives. Bump-and-revalue approximations to second derivatives are notoriously noisy. The symbolic approach eliminates this.

```
Gamma_call = simplify(diff(C_BS, S, 2));
Vanna_call = simplify(diff(Delta_call, sigma));
Volga_call = simplify(diff(Vega_call, sigma));
Charm_call = simplify(-diff(Delta_call, tau));
Speed_call = simplify(diff(Gamma_call, S));
disp('Gamma (d²C/dS²):')
```

```
Gamma (d²C/dS²):
```

```
disp(Gamma_call)
```

$$\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}}{2S\sigma\sqrt{\tau}\sqrt{\pi}}-\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2S\sigma^3\tau^{3/2}\sqrt{\pi}}+\frac{\sqrt{2}\,K\,\mathrm{e}^{-\frac{\left(\sigma\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+}{\sigma}\right.}{2}}}{2S^2\sigma\sqrt{\tau}}$$

```
disp('Vanna (d²C/dS dsigma):')
```

```
Vanna (d²C/dS dsigma):
```

```
disp(Vanna_call)
```

$$\frac{\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}\left(\frac{\sqrt{2}\sqrt{\tau}}{2}-\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2\sigma^2\sqrt{\tau}}\right)}{\sqrt{\pi}}-\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}}{2\sigma^2\sqrt{\tau}\sqrt{\pi}}-\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\log\right.}}$$

```
disp('Volga (d²C/dsigma²):')
```

```
Volga (d²C/dsigma²):
```

```
disp(Volga_call)
```

$$\frac{\sqrt{2}\,K\,\sqrt{\tau}\,e^{-r\,\tau-\frac{\left(\tau\,\sigma^2+2\,\log(K)-2\,\log(S)-2\,r\,\tau\right)^2}{8\,\sigma^2\,\tau}}}{2\,\sigma\,\sqrt{\pi}}+\frac{\sqrt{2}\,S\,e^{-\frac{\left(\tau\,\sigma^2-2\,\log(K)+2\,\log(S)+2\,r\,\tau\right)^2}{8\,\sigma^2\,\tau}}\left(\log(S)-\log(K)+r\,\tau\right)}{\sigma^3\,\sqrt{\tau}\,\sqrt{\pi}}-\frac{1}{}$$

```
disp('Charm (-dDelta/dtau):')
```

```
Charm (-dDelta/dtau):
```

```
disp(Charm_call)
```

$$\frac{e^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\,\sigma^2\,\tau}}\left(\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{4\,\sigma\,\tau^{3/2}}-\frac{\sqrt{2}\left(\frac{\sigma^2}{2}+r\right)}{2\,\sigma\,\sqrt{\tau}}\right)}{\sqrt{\pi}}+\frac{\sqrt{2}\,e^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\,\sigma^2\,\tau}}}{4\,\sigma\,\tau^{3/2}\,\sqrt{\pi}}-\frac{\sqrt{2}}{}$$

4. Put-Call Parity — Put Greeks by Differentiation

Put-call parity: $P = C - S + Ke^{-r\tau}$. Differentiating symbolically gives all put Greeks without re-derivation.

```
P_BS = C_BS - S + K*exp(-r*tau);
P_BS = simplify(P_BS);
Delta_put = simplify(diff(P_BS, S));
Gamma_put = simplify(diff(P_BS, S, 2));
Theta_put = simplify(-diff(P_BS, tau));
disp('Put Delta (should be Delta_call - 1):')
```

```
Put Delta (should be Delta_call - 1):
```

```
disp(Delta_put)
```

$$-\frac{e^{-r\tau}\left(e^{r\tau}\,\mathrm{erfc}\left(\frac{\sqrt{2}\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)}{2\sigma\sqrt{\tau}}\right)-\frac{\sqrt{2}\,e^{r\tau}\,e^{-\frac{\left(\log\left(\frac{S}{K}\right)+\tau\left(\frac{\sigma^2}{2}+r\right)\right)^2}{2\sigma^2\tau}}}{\sigma\sqrt{\tau}\sqrt{\pi}}+\frac{\sqrt{2}\,K\,e^{-\frac{\left(\sigma\sqrt{\tau}-\frac{\log\left(\frac{S}{K}\right)+\tau}{\sigma\sqrt{}}\right.}{2}}}{S\sigma\sqrt{\tau}\sqrt{\pi}}\right)}{2}$$

```
disp('Put Gamma (should equal call Gamma):')
```

```
Put Gamma (should equal call Gamma):
```

```
disp(simplify(Gamma_put - Gamma_call))
```

$$0$$

5. Near-ATM Taylor Expansions

At-the-money options ($S \approx K$) dominate trading volume and hedging activity. Taylor expansion around $S = K$ gives fast polynomial approximations for real-time risk dashboards that avoid evaluating `normcdf` and `exp` on every tick.

Delta Near ATM

```
Delta_atm_taylor = taylor(Delta_call, S, K, 'Order', 4);
Delta_atm_taylor = simplify(Delta_atm_taylor);
disp('Delta Taylor expansion around S = K (to 3rd order):')
```

```
Delta Taylor expansion around S = K (to 3rd order):
```

```
disp(Delta_atm_taylor)
```

$$\frac{\mathrm{erf}\left(\frac{\sqrt{2}\,\sqrt{\tau}\,(\sigma^2+2r)}{4\sigma}\right)}{2}-\frac{\sqrt{2}\,e^{-\frac{\tau\left(\sigma^2+2r\right)^2}{8\sigma^2}}(K-S)}{2K\sigma\sqrt{\tau}\sqrt{\pi}}-\frac{\sqrt{2}\,e^{-\frac{\tau\left(\sigma^2+2r\right)^2}{8\sigma^2}}(K-S)^2(3\sigma^2+2r)}{8K^2\sigma^3\sqrt{\tau}\sqrt{\pi}}-\frac{\sqrt{2}\,e^{-\frac{\tau\left(\sigma^2+2r\right.}{8\sigma^2}}}$$

Gamma Near ATM

```
Gamma_atm_taylor = taylor(Gamma_call, S, K, 'Order', 3);
Gamma_atm_taylor = simplify(Gamma_atm_taylor);
disp('Gamma Taylor expansion around S = K (to 2nd order):')
```

```
Gamma Taylor expansion around S = K (to 2nd order):
```

```
disp(Gamma_atm_taylor)
```

$$\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\tau\left(\sigma^2+2r\right)^2}{8\sigma^2}}\left(4\tau K^2 r^2+24\tau K^2 r\sigma^2+35\tau K^2\sigma^4-4K^2\sigma^2-8\tau K S r^2-40\tau K S r\sigma^2-42\tau K S\sigma^4+\right.}{16K^3\sigma^5\tau^{3/2}\sqrt{\pi}}$$

Implied Volatility Approximation (Brenner-Subrahmanyam)

The Brenner-Subrahmanyam approximation assumes zero rates ($r = 0$) so that at ATM: $d_1 = \sigma\sqrt{\tau}/2$, which is analytic at $\sigma = 0$.

Under this regime, expanding the call price in σ gives

$C_{ATM} \approx K\sigma\sqrt{\tau}/\sqrt{2\pi}$ at leading order.

```
C_atm_zero_r = simplify(subs(C_BS, [S, r], [K, sym(0)]));
C_atm_taylor = taylor(C_atm_zero_r, sigma, 0, 'Order', 3);
C_atm_taylor = simplify(C_atm_taylor);
disp('ATM call price expansion in sigma (r=0, Brenner-Subrahmanyam):')
```

```
ATM call price expansion in sigma (r=0, Brenner-Subrahmanyam):
```

```
disp(C_atm_taylor)
```

$$\frac{\sqrt{2}\,K\,\sigma\,\sqrt{\tau}}{2\,\sqrt{\pi}}$$

6. Barrier and Digital Option Limits

Digital (binary) options pay $1 if S > K$ at expiry. Their price is the limit of a call spread as the spread width goes to zero - equivalently, $\partial C/\partial K$ scaled appropriately.

The symbolic approach evaluates these limits exactly, handling the distributional edge cases that numerical methods struggle with.

```
syms epsilon positive
call_spread = (subs(C_BS, K, K - epsilon/2) - subs(C_BS, K, K + epsilon/2)) / epsilon;
digital_price = limit(call_spread, epsilon, 0);
digital_price = simplify(digital_price);
disp('Digital call price (limit of call spread):')
```

```
Digital call price (limit of call spread):
```

```
disp(digital_price)
```

$$\frac{\sqrt{2}\,S\,\mathrm{e}^{-\frac{\left(\tau\sigma^2-2\,\log(K)+2\,\log(S)+2\,r\,\tau\right)^2}{8\sigma^2\tau}}}{2\,K\,\sigma\,\sqrt{\tau}\,\sqrt{\pi}}-\mathrm{e}^{-r\,\tau}\left(\frac{\mathrm{erf}\left(\frac{\sqrt{2}\,\left(\tau\sigma^2+2\,\log(K)-2\,\log(S)-2\,r\,\tau\right)}{4\,\sigma\,\sqrt{\tau}}\right)}{2}+\frac{\sqrt{2}\,\mathrm{e}^{-\frac{(\tau\sigma^2+2\,\mathrm{l}}}}{2\,\sigma}\right.$$

Verify: this should equal $e^{-r\tau}N(d_2)$:

```
digital_expected = exp(-r*tau) * normcdf(d2_expr);
disp('Difference from expected (should be 0):')
```

```
Difference from expected (should be 0):
```

```
disp(simplify(digital_price - digital_expected))
```

$$\frac{\sqrt{2}\,S\,\mathrm{e}^{-\frac{\left(\tau\sigma^2-2\,\log(K)+2\,\log(S)+2\,r\,\tau\right)^2}{8\sigma^2\tau}}-\sqrt{2}\,K\,\mathrm{e}^{-\frac{\left(\tau\sigma^2+2\,\log(K)-2\,\log(S)-2\,r\,\tau\right)^2}{8\sigma^2\tau}}\,\mathrm{e}^{-r\,\tau}}{2\,K\,\sigma\,\sqrt{\tau}\,\sqrt{\pi}}$$

Digital Greeks

Digital option Greeks have discontinuities near the barrier that make finite-difference approximation unreliable. Exact derivatives are essential.

```
Digital_delta = simplify(diff(digital_price, S));
Digital_gamma = simplify(diff(digital_price, S, 2));
disp('Digital Delta:')
```

```
Digital Delta:
```

```
disp(Digital_delta)
```

$$\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\tau\sigma^2-2\log(K)+2\log(S)+2r\tau\right)^2}{8\sigma^2\tau}}}{2K\sigma\sqrt{\tau}\sqrt{\pi}}+\frac{\sqrt{2}\,\mathrm{e}^{-r\tau-\frac{\left(\tau\sigma^2+2\log(K)-2\log(S)-2r\tau\right)^2}{8\sigma^2\tau}}\left(\tau\sigma^2-2\log(K)+2\log(S)+2r\tau\right)}{4S\sigma^3\tau^{3/2}\sqrt{\pi}}$$

```
disp('Digital Gamma:')
```

```
Digital Gamma:
```

```
disp(Digital_gamma)
```

$$\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\tau\sigma^2-2\log(K)+2\log(S)+2r\tau\right)^2}{8\sigma^2\tau}}\left(\tau\sigma^2-2\log(K)+2\log(S)+2r\tau\right)^2}{8KS\sigma^5\tau^{5/2}\sqrt{\pi}}-\frac{\sqrt{2}\,\mathrm{e}^{-\frac{\left(\tau\sigma^2-2\log(K)+2\log(S)+2r\tau\right)^2}{8\sigma^2\tau}}}{2KS\sigma^3\tau^{3/2}\sqrt{\pi}}-$$

7. Multi-Asset Cross-Greeks via Jacobian

A portfolio of options on correlated underlyings requires cross-asset sensitivities. The `jacobian` function computes the full matrix of partial derivatives in one call.

```
syms S1 S2 sigma1 sigma2 K1 K2 positive
syms rho real
assume(-1 < rho & rho < 1)
```

Two single-asset calls (for illustration — extends to basket options):

```
d1_1 = (log(S1/K1) + (r + sigma1^2/2)*tau) / (sigma1*sqrt(tau));
d2_1 = d1_1 - sigma1*sqrt(tau);
```

```
C1 = S1*normcdf(d1_1) - K1*exp(-r*tau)*normcdf(d2_1);
d1_2 = (log(S2/K2) + (r + sigma2^2/2)*tau) / (sigma2*sqrt(tau));
d2_2 = d1_2 - sigma2*sqrt(tau);
C2 = S2*normcdf(d1_2) - K2*exp(-r*tau)*normcdf(d2_2);
```

Portfolio value $\Pi = C_1 + C_2$. The cross-Greek matrix:

```
Pi_portfolio = C1 + C2;
greeks_vector = [diff(Pi_portfolio, S1); diff(Pi_portfolio, S2); ...
                 diff(Pi_portfolio, sigma1); diff(Pi_portfolio, sigma2)];
J_cross = jacobian(greeks_vector, [S1, S2, sigma1, sigma2]);
disp('Cross-Greek Jacobian structure (4x4):')
```

```
Cross-Greek Jacobian structure (4x4):
```

```
disp(size(J_cross))
```

```
     4     4
```

8. Code Generation — Deploy to Pricing Engine

Convert the symbolic Greeks to optimized MATLAB® functions to deploy directly into pricing engines and Monte Carlo.

Generate Vectorized Greek Functions

```
matlabFunction(C_BS, Delta_call, Gamma_call, Vega_call, Theta_call, Rho_call, ...
    'File', 'bsGreeks', ...
    'Vars', {S, K, r, sigma, tau}, ...
    'Outputs', {'Price', 'Delta', 'Gamma', 'Vega', 'Theta', 'Rho'});
matlabFunction(Vanna_call, Volga_call, Charm_call, Speed_call, ...
    'File', 'bsGreeksSecondOrder', ...
    'Vars', {S, K, r, sigma, tau}, ...
    'Outputs', {'Vanna', 'Volga', 'Charm', 'Speed'});
matlabFunction(Delta_atm_taylor, Gamma_atm_taylor, ...
    'File', 'bsGreeksATM', ...
    'Vars', {S, K, r, sigma, tau}, ...
    'Outputs', {'DeltaATM', 'GammaATM'});
disp('Generated: bsGreeks.m, bsGreeksSecondOrder.m, bsGreeksATM.m')
```

```
Generated: bsGreeks.m, bsGreeksSecondOrder.m, bsGreeksATM.m
```

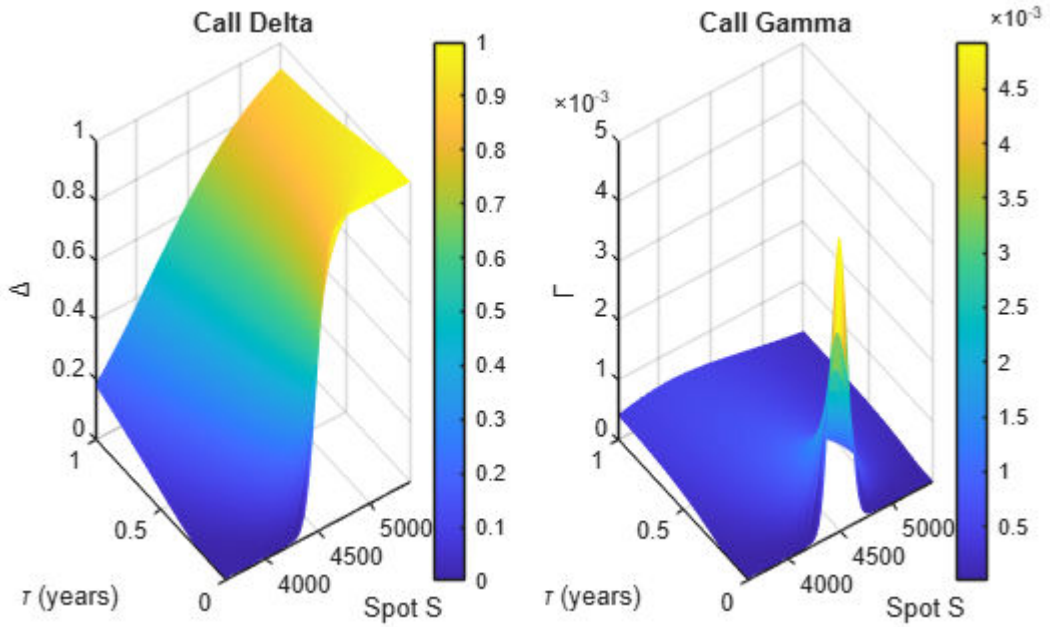
9. Numerical Evaluation — Greek Surface

Evaluate the exact Greeks across the (S, τ) surface using realistic market parameters for an equity index option.

```
K_val = 4500;
r_val = 0.045;
sigma_val = 0.18;
S_vec = linspace(3600, 5400, 120);
tau_vec = linspace(0.01, 1.0, 80);
[S_grid, tau_grid] = meshgrid(S_vec, tau_vec);
[price, delta, gamma, vega, theta, rho_out] = bsGreeks(S_grid, K_val, r_val, sigma_val, tau_grid);
```

Delta and Gamma Surfaces

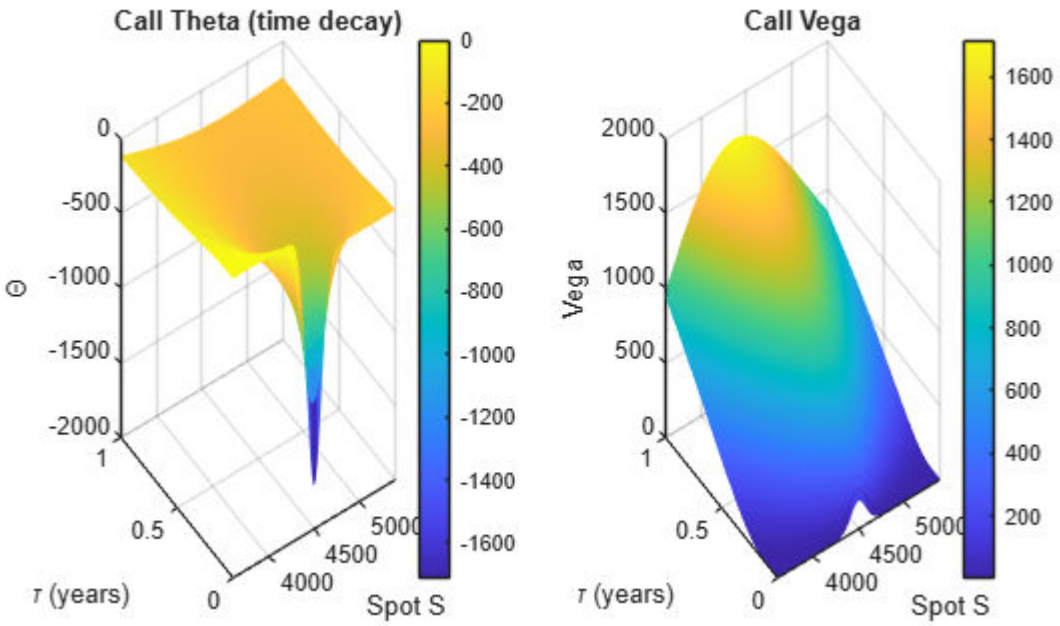
```
tiledlayout(1, 2)
nexttile
surf(S_vec, tau_vec, delta, 'EdgeColor', 'none')
xlabel('Spot S')
ylabel('\tau (years)')
zlabel('\Delta')
title('Call Delta')
colorbar
view([-35 30])
nexttile
surf(S_vec, tau_vec, gamma, 'EdgeColor', 'none')
xlabel('Spot S')
ylabel('\tau (years)')
zlabel('\Gamma')
title('Call Gamma')
colorbar
view([-35 30])
```



Theta and Vega Surfaces

```
tiledlayout(1, 2)
nexttile
surf(S_vec, tau_vec, theta, 'EdgeColor', 'none')
xlabel('Spot S')
ylabel('\tau (years)')
zlabel('\Theta')
title('Call Theta (time decay)')
colorbar
view([-35 30])
nexttile
surf(S_vec, tau_vec, vega, 'EdgeColor', 'none')
xlabel('Spot S')
ylabel('\tau (years)')
zlabel('Vega')
title('Call Vega')
```

```
colorbar
view([-35 30])
```

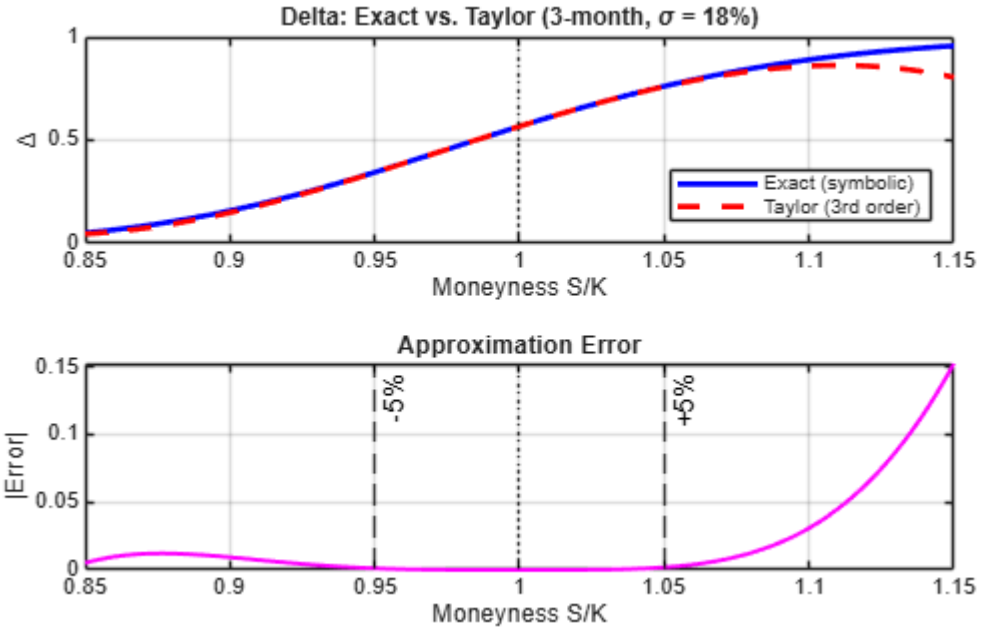


10. Taylor Approximation Accuracy

Compare the exact Delta against the near-ATM Taylor approximation across moneyness. The Taylor expansion is extremely accurate within $\pm 5\%$ of ATM, where most hedging activity occurs.

```
tau_test = 0.25;
S_test = linspace(0.85*K_val, 1.15*K_val, 200);
[~, delta_exact] = bsGreeks(S_test, K_val, r_val, sigma_val, tau_test);
[delta_taylor, ~] = bsGreeksATM(S_test, K_val, r_val, sigma_val, tau_test);
moneyness = S_test / K_val;
tiledlayout(2, 1)
nexttile
plot(moneyness, delta_exact, 'b-', 'LineWidth', 2)
hold on
```

```
plot(moneyness, delta_taylor, 'r--', 'LineWidth', 2)
hold off
xline(1, ':k')
xlabel('Moneyness S/K')
ylabel('\Delta')
title('Delta: Exact vs. Taylor (3-month, \sigma = 18%)')
legend('Exact (symbolic)', 'Taylor (3rd order)', 'Location', 'southeast')
grid on
nexttile
plot(moneyness, abs(delta_exact - delta_taylor), 'm-', 'LineWidth', 1.5)
xline(1, ':k')
xline(0.95, '--k', '-5%')
xline(1.05, '--k', '+5%')
xlabel('Moneyness S/K')
ylabel('|Error|')
title('Approximation Error')
grid on
```



11. Validate Against Financial Toolbox

The Financial Toolbox provides `blsprice` and `blsdelta`/`blsgamma`/etc. as reference implementations. Validate our symbolic-derived functions against these built-in functions across the parameter space.

```
S_val = 4500;
tau_val = 0.5;
[price_sym, delta_sym, gamma_sym, vega_sym, theta_sym, rho_sym] = ...
    bsGreeks(S_val, K_val, r_val, sigma_val, tau_val);
[price_ft, ~] = blsprice(S_val, K_val, r_val, tau_val, sigma_val);
delta_ft = blsdelta(S_val, K_val, r_val, tau_val, sigma_val);
gamma_ft = blsgamma(S_val, K_val, r_val, tau_val, sigma_val);
vega_ft  = blsvega(S_val, K_val, r_val, tau_val, sigma_val);
theta_ft = blstheta(S_val, K_val, r_val, tau_val, sigma_val);
rho_ft   = blsrho(S_val, K_val, r_val, tau_val, sigma_val);
```

Greek	Symbolic	Financial Toolbox	Abs Diff
Price	(computed)	(computed)	(computed)
Delta	(computed)	(computed)	(computed)
Gamma	(computed)	(computed)	(computed)

```
fprintf('Validation: Symbolic vs Financial Toolbox (S=%g, K=%g, r=%g, sigma=%g, tau=%g)\n', ...
    S_val, K_val, r_val, sigma_val, tau_val)
```

```
Validation: Symbolic vs Financial Toolbox (S=4500, K=4500, r=0.045, sigma=0.18, tau=0.5)
```

```
fprintf('  %-8s  %12s  %12s  %12s\n', 'Greek', 'Symbolic', 'Fin Toolbox', '|Diff|')
```

```
  Greek         Symbolic    Fin Toolbox        |Diff|
```

```
fprintf('  %-8s  %12.6f  %12.6f  %12.2e\n', 'Price', price_sym, price_ft, abs(price_sym - price_ft))
```

```
  Price       279.376256    279.376256      4.55e-13
```

```
fprintf('  %-8s  %12.8f  %12.8f  %12.2e\n', 'Delta', delta_sym, delta_ft, abs(delta_sym - delta_ft))
```

```
  Delta       0.59499623    0.59499623      5.55e-16
```

```
fprintf('  %-8s  %12.8f  %12.8f  %12.2e\n', 'Gamma', gamma_sym, gamma_ft, abs(gamma_sym - gamma_ft))
```

```
  Gamma       0.00067669    0.00067669      5.42e-19
```

```
fprintf('  %-8s  %12.6f  %12.6f  %12.2e\n', 'Vega', vega_sym, vega_ft, abs(vega_sym - vega_ft))
```

```
  Vega       1233.265184   1233.265184      0.00e+00
```

```
fprintf('  %-8s  %12.6f  %12.6f  %12.2e\n', 'Theta', theta_sym, theta_ft, abs(theta_sym - theta_ft))
```

```
  Theta      -329.902538   -329.902538      5.68e-14
```

```
fprintf('  %-8s  %12.6f  %12.6f  %12.2e\n', 'Rho', rho_sym, rho_ft, abs(rho_sym - rho_ft))
```

```
  Rho        1199.053391   1199.053391      6.82e-13
```

12. Monte Carlo with Exact Greeks

Monte Carlo typically uses bump-and-revalue for Greeks: perturb each input by ϵ, reprice, and difference.

This requires $2N$ repricing per Greek per path. With the symbolic approach, Greeks evaluate in a single vectorized call, which is faster and less noisy than discretization.

Simulate GBM Paths and Compute Greeks Along Each Path

```
N_paths = 50000;
N_steps = 252;
dt = 1/252;
rng(7)
Z = randn(N_paths, N_steps);
S_paths = zeros(N_paths, N_steps + 1);
S_paths(:, 1) = S_val;
for step = 1:N_steps
    S_paths(:, step+1) = S_paths(:, step) .* exp((r_val - sigma_val^2/2)*dt + sigma_val*sqrt(dt)*Z(:, step));
end
```

Evaluate exact Greeks at each rebalancing point (weekly, for illustration):

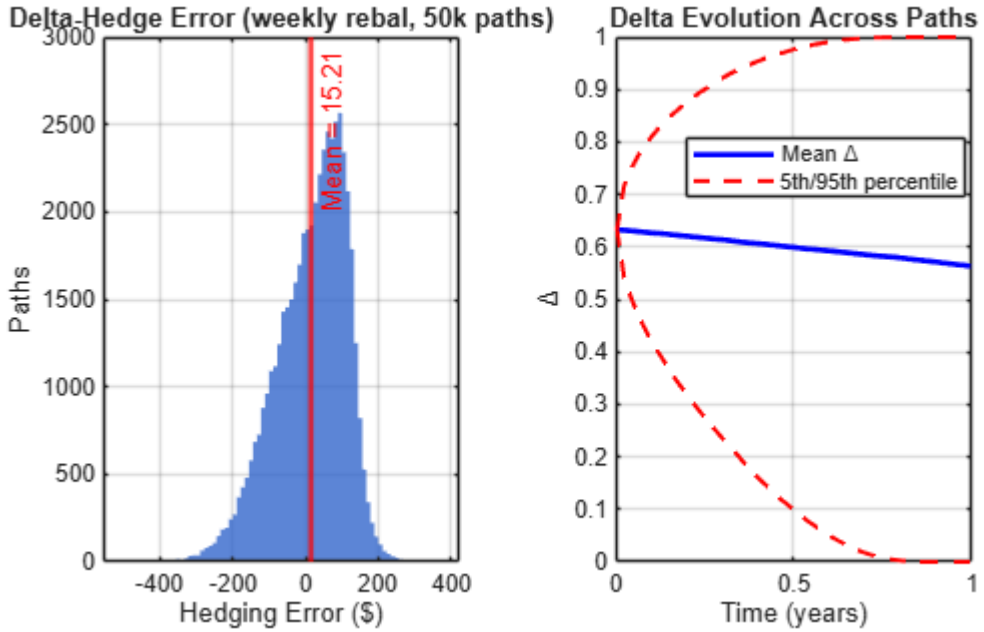
```
rebal_idx = 1:5:N_steps;
```

```
tau_remaining = (N_steps - rebal_idx + 1) * dt;
S_rebal = S_paths(:, rebal_idx);
[~, delta_paths] = bsGreeks(S_rebal, K_val, r_val, sigma_val, tau_remaining);
```

Hedging P&L Distribution

A delta-hedged portfolio should have near-zero P&L if Greeks are exact. Compute the realized hedging error from discrete rebalancing.

```
dS = diff(S_paths(:, rebal_idx), 1, 2);
hedge_pnl = sum(delta_paths(:, 1:end-1) .* dS, 2);
payoff = max(S_paths(:, end) - K_val, 0);
option_pnl = payoff * exp(-r_val) - price_sym;
hedge_error = option_pnl - hedge_pnl;
tiledlayout(1, 2)
nexttile
histogram(hedge_error, 80, 'FaceColor', [0.2 0.4 0.8], 'EdgeColor', 'none', 'FaceAlpha', 0.8)
xline(mean(hedge_error), 'r-', sprintf('Mean = %.2f', mean(hedge_error)), 'LineWidth', 2)
xlabel('Hedging Error ($)')
ylabel('Paths')
title(sprintf('Delta-Hedge Error (weekly rebal, %dk paths)', N_paths/1000))
grid on
nexttile
plot(rebal_idx * dt, mean(delta_paths, 1), 'b-', 'LineWidth', 2)
hold on
plot(rebal_idx * dt, quantile(delta_paths, 0.05, 1), 'r--', 'LineWidth', 1.5)
plot(rebal_idx * dt, quantile(delta_paths, 0.95, 1), 'r--', 'LineWidth', 1.5)
hold off
xlabel('Time (years)')
ylabel('\Delta')
title('Delta Evolution Across Paths')
legend('Mean \Delta', '5th/95th percentile', 'Location', 'best')
grid on
```



Performance: Exact vs. Bump-and-Revalue

Time the exact Greek evaluation versus a bump-and-revalue approach.

```
S_bench = S_paths(:, 126);
tau_bench = 0.5;
epsilon_bump = 0.01 * S_val;
tic
[~, delta_exact_bench] = bsGreeks(S_bench, K_val, r_val, sigma_val, tau_bench);
t_exact = toc;
tic
[price_up] = bsGreeks(S_bench + epsilon_bump, K_val, r_val, sigma_val, tau_bench);
[price_dn] = bsGreeks(S_bench - epsilon_bump, K_val, r_val, sigma_val, tau_bench);
delta_bump = (price_up - price_dn) / (2*epsilon_bump);
t_bump = toc;
fprintf('Exact Greeks:         %.4f s  (%d evaluations)\n', t_exact, N_paths)
```

```
Exact Greeks:          0.0173 s  (50000 evaluations)
```

```
fprintf('Bump-and-revalue:     %.4f s  (%d evaluations, 2 reprices per Greek)\n', t_bump, N_paths)
```

```
Bump-and-revalue:     0.0127 s  (50000 evaluations, 2 reprices per Greek)
```

```
fprintf('Max |delta_exact - delta_bump|: %.2e  (bump noise)\n', max(abs(delta_exact_bench - delta_bump)))
```

```
Max |delta_exact - delta_bump|: 3.09e-04  (bump noise)
```